

Autonomous Cleaning Robot with Debris Classification Feature using Advanced ML Algorithms

Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Technology

in

Mechanical Engineering

Submitted by

Siddartha Royal G (Roll No: 20MCE1009)

Jayanth Kumar M S (Roll No: 20MCE1011)

Jayanth Narayana K (Roll No: 20MCE1013)

Under the Supervision of

Dr. Gaurang Ruhela

Assistant Professor



Department of Mechanical Engineering

National Institute of Technology Goa

May, 2024



Department of Mechanical Engineering
National Institute of Technology Goa
Cuncolim, Goa, India - 403407

CERTIFICATE

This is to certify that the work contained in this report entitled “**Autonomous Cleaning Robot with debris classification feature using advanced ML Algorithms**” is submitted by the group members Mr. Siddartha Royal (Roll No: 20MCE1009), Mr. Jayanth Kumar (Roll No: 20MCE1011) and Mr. Jayanth Narayana (Roll No: 20MCE1013) to the Department of Mechanical Engineering, National Institute of Technology Goa, for the partial fulfillment of the requirements for the degree of **Bachelor of Technology in Mechanical Engineering**.

They have carried out their work under my supervision. This work has not been submitted elsewhere for the award of any other degree or diploma.

The project work, in our opinion, has reached the standard fulfilling of the requirements for the degree of Bachelor of Technology in Mechanical Engineering in accordance with the regulations of the Institute.

Dr. Gaurang Ruhela (Supervisor)

Department of MCE
NIT Goa

Dr. B. Santhi (HOD)

Department of MCE
NIT Goa

Acknowledgments

We wish to express our deepest gratitude to all those who have extended their cooperation in various capacities throughout the execution of our project. It is with sincere appreciation that we acknowledge the invaluable assistance provided by all those individuals.

We extend our heartfelt thanks to Dr. Gaurang Ruhela, faculty member of the Mechanical Engineering department, for generously dedicating his valuable time to review and analyze our project at every stage. We consider ourselves incredibly fortunate to have had the opportunity to collaborate with him.

Contents

Acknowledgments	2
1 Introduction	8
1.1 Background and Motivation	8
1.2 Project Objective	8
1.3 Scope and Significance	9
1.4 Related Work	9
1.5 Our Work Plan	10
2 Literature Review	11
3 Design and Hardware Components	19
3.1 Robot Layout Design	19
3.1.1 Chassis Layout	20
3.1.2 Cleaning Mechanism	21
3.1.3 Camera Placement	21
3.2 Components Required	22
3.2.1 Arduino Mega	22
3.2.2 Motor Driver and Motor Shield	23
3.2.3 BLDC Motor and ESC	24
3.2.4 Servomotor	24
3.2.5 ESP 32 Camera module	25
3.2.6 Power Supply	25

4	Machine Learning Model	27
4.1	Dataset Preparation	27
4.2	Model Architecture	27
4.3	Training Process	28
4.3.1	Model Preparation	28
4.3.2	Accuracy Results	30
4.4	Evaluation Metrics	32
4.5	Model Optimization	32
4.6	Quantization-Aware Training	32
4.7	TensorFlow Lite Conversion	33
4.8	Code Implementation for Pruning	33
4.8.1	Testing	34
4.8.2	Quantized Model	35
4.8.3	Results	35
4.9	Integration with Edge Impulse	36
4.10	Further Optimization and Quantization	36
4.11	Library Download for ESP32 CAM	36
5	Arduino Software and Robot Assembly	37
5.1	Hardware Configuration	37
5.2	Arduino Software Architecture	39
5.2.1	Sensor Interface	39
5.2.2	Actuator Control	39
5.2.3	Communication	39
5.3	Arduino Code Implementation	39
5.3.1	Arduino Mega	39
5.3.2	ESP 32 code	43
5.4	Robot Assembly and Integration	48
6	Conclusion	50
7	References	52

List of Figures

3.1	Basic Robot Layout	19
3.2	Top View of Robot	20
3.3	Camera Module Placement	22
3.4	Arduino Mega 2560	23
3.5	L293D Motor Shield	24
3.6	L298N Motor Driver	24
3.7	BLDC Motor	24
3.8	Electronic Speed Controller	24
3.9	SG90	25
3.10	MG985	25
3.11	ESP 32 Camera Module	25
3.12	9V Battery	26
3.13	Li-ion Battery	26
4.1	CNN Model Code in VSCODE	29
4.2	Features Map	30
4.3	Training Results with Edge Impulse Plotter	30
4.4	Testing Results with Edge Impulse Plotter	31
4.5	Terminal Result with Confusion Matrix	31
4.6	Data Quantisation Code	34
4.7	Testing with Image from System	34
4.8	Training Performance of Quantized Model(int8)	35
4.9	Result from Previous Code	35

5.1	Arduino Circuit	38
5.2	Arduino Mega Code	42
5.3	ESP 32 Code	47
5.4	Top View of Robot and BLDC Motor with Vacuum Chamber	48
5.5	Side View of Robot with Suction Pipe	49
5.6	Top View of Robot with Camera Module	49

List of Tables

3.1	List of Components	26
-----	------------------------------	----

Chapter 1

Introduction

1.1 Background and Motivation

The motivation behind creating an autonomous cleaning robotic systems comes from the need for innovative solutions in household maintenance. Inspired by the increasing demands for smart and efficient technologies in this field, this project aims to develop a robot capable of autonomously navigating and cleaning indoor spaces. The background lies in addressing the challenges of modern lifestyles, where time for household chores is limited. By leveraging advancements in robotics, machine learning, and computer vision, this initiative seeks to offer a convenient, time-saving, and effective solution for maintaining cleanliness in homes. The project aligns with the quest for smarter, more adaptable technologies that enhance daily living, promoting convenience and efficiency in domestic environments.

1.2 Project Objective

The project's primary objective is to develop an intelligent cleaning robot integrating machine learning, TinyML, and advanced path planning techniques. This robot will excel in recognizing and categorizing debris, enhancing cleaning efficiency in diverse environments. The overarching aim is to out

perform traditional cleaning robots by leveraging cutting-edge technologies, ensuring adaptability to real-world challenges, and elevating overall cleanliness standards in various settings.

1.3 Scope and Significance

The scope of this project encompasses the development of an autonomous cleaning robot capable of navigating various indoor environments, detecting and addressing cleaning needs intelligently. By integrating sensors, machine learning algorithms, and efficient control mechanisms, the robot aims to cover a wide range of surfaces, identify debris, and execute cleaning tasks efficiently on edgy processors. Our cleaning robot surpasses Roomba’s capabilities through advanced technologies. Unlike Roomba’s random cleaning pattern, our robot employs machine learning, computer vision, and optimized path planning for systematic and adaptive cleaning. It detects and classifies debris, adjusting cleaning methods based on real-time conditions. This innovation enhances efficiency and adaptability, ensuring a more thorough and tailored cleaning experience. The integration of these technologies sets our robot apart, elevating its intelligence and performance beyond the standard set by traditional robotic vacuum cleaners like Roomba.

1.4 Related Work

A variety of cutting-edge deep learning techniques have advanced the capabilities of cleaning robots, revolutionizing household cleaning tasks. Mbunge [1] proposed a hybrid model for face mask detection during the COVID-19 pandemic, achieving a remarkable testing accuracy of 99.64%. Zhao and Zhong-Qiu [2] introduced the R-CNN framework for object detection, followed by the fast R-CNN and YOLO models, which significantly improved detection speed and accuracy.

Ferentinos [3] developed a semi-automatic system for detecting soybean leaf diseases with nearly 90% overall accuracy using Support Vector Machine (SVM). Zech [4] evaluated pneumonia-screening Convolutional Neural Networks (CNNs) across multiple institutions, highlighting challenges in generalizing models across different datasets. Sanker [5] proposed a k-means classifier for plant leaf disease detection, while Venu [6] discussed the ESP32 camera’s versatility in robotic projects, emphasizing its real-time processing capabilities and wireless communication features.

Dong [14] have carried out classification using transfer learning techniques and achieved 88.90 accuracy and Warden [17] built a ML model and implemented that on edged processors like arduino UNO using camera module and TFLite arduino library. A., Paul [13] used edge impulse software to build a custom library for any processor for running ML algorithm in edgy devices, he used it on ESP32 cam module. while this can be pulled off by eloquentML library which is inbuilt in Arduino IDE which was done by Sudharshan [20].

1.5 Our Work Plan

After researching all these paper work and journals, we have decide to built a model from already pretrained model, and tested out with ‘MobileNETV2’ and ‘SqueezeNET’ and opted for primer one because of its accuracy and least computationl power. we selected ESP32 cam module for real time debris classification and EDGE IMPULSE for custom library creation and optimum quantisization. Then generating serial communication between Cam module and Arduino mega, which will activate the respective cleaning mechanism based on the input from cam.

Chapter 2

Literature Review

1. Mbunge et al. [1] proposed a hybrid model combining SVM, decision trees, and ensemble methods. While achieving a remarkable testing accuracy of 99.64%, the potential heaviness of multiple clusters renders it unsuitable for deployment on low-end devices. Furthermore, the authors emphasized the importance of dataset authenticity, underscoring the need for real-world face mask images to enhance the precision and real-world applicability of detection systems.
2. Zhao et al. [2] R-CNN framework marked a breakthrough in object detection, pioneering CNN features for region-based detection. Their fast R-CNN model streamlined detection by integrating proposal generation and feature extraction, setting new speed and accuracy benchmarks. Zhong-Qiu's YOLO model further revolutionized detection with real-time performance and single-stage architecture, predicting bounding boxes and class probabilities simultaneously. This innovation showcased the potential for efficient and accurate object detection across diverse applications.
3. Ferentinos et al. [3] devised a semi-automatic system for detecting soy-

bean leaf diseases and predicting their severity. Their dataset comprises 4775 images categorized into four classes. Leveraging Support Vector Machine (SVM) as a classifier, they achieved an impressive overall accuracy of nearly 90%. Nonetheless, the potential heaviness of multiple clusters within this model might render it unsuitable for deployment on low-end devices.

4. Zech et al. [4] evaluated pneumonia-screening CNNs across three institutions: NIH, MSH, and IU, using 158,323 chest radiographs. Variations in pneumonia prevalence were noted. Models trained on NIH or MSH data performed similarly on IU data but showed lower performance on each other's data. Combining MSH and NIH data yielded high internal performance but didn't generalize to IU. Pooling data from sites with different pneumonia prevalence improved internal but not external performance. CNNs accurately identified hospital systems and departments, suggesting potential biases. Internal performance consistently exceeded external, highlighting challenges in generalizing CNN models across institutions.
5. Sanker [5] initially proposed a k-means classifier for plant leaf disease detection but lacked crucial dataset information and accuracy. In a subsequent study, they enhanced the concept by developing a semi-automatic system for soybean leaf detection, offering severity prediction. Their dataset comprised 4775 images across four classes. Employing SVM, they attained nearly 90% overall accuracy. However, the model's multiple clusters could render it bulky and unsuitable for low-end devices, raising concerns about its practicality for real-life plant disease diagnosis.
6. Venu, D. N [6] outlines the ESP32 camera's utility in robotics, citing

its Wi-Fi and Bluetooth capabilities for wireless communication and sensor integration. With motor driver compatibility, it enables precise movement control, aided by its dual-core processor for real-time data processing. The ESP32’s ability to host a web server allows remote control via a user-friendly interface. Despite its versatility, challenges like real-world testing and application-specific optimization persist, as noted by Sanker [20]. Nonetheless, the ESP32 remains a potent platform for varied robotic applications.

7. The study by Balakrishnan Ramalingam et al. [7] in Applied Sciences introduces a cascaded machine-learning approach for debris classification in floor-cleaning robots. Combining deep Convolutional Neural Network (CNN) with Support Vector Machine (SVM), it achieves 95.5 percentage accuracy in identifying solid and liquid debris, surpassing existing techniques, with potential real-world deployment advantages.
8. Jia Yin et al. [8] present a method for table cleaning and inspection using a Human Support Robot (HSR) in a food court setting. Their lightweight Deep Convolutional Neural Network (DCNN) achieves 96 percentage accuracy in food litter recognition. The planner framework generates efficient cleaning paths, enhancing HSR’s real-world applicability for cleanliness inspections and diverse cleaning modes.
9. Balakrishnan Ramalingam et al. [9] optimized convolutional neural network (CNN) layers to reduce detection time and expanding obstacle and debris classes for safer navigation and more efficient cleaning on staircases using sTetro staircase maintenance robot. The research presents a machine learning-based operational framework demonstrating real-time object detection and localization for staircases, obstacles, and debris. This framework has practical implications for automating

and improving the efficiency of staircase cleaning tasks in multistory buildings.

10. Soowook Choe et al. [10] explored the combination of probabilistic reasoning and deep learning for indoor place category recognition in cleaning robots. The study demonstrates that this approach provides an effective solution with generality and superior performance, making it promising for various applications beyond cleaning robots. Further research is recommended for scenarios involving RGBD cameras.
11. Sanchez-Iborra et al. [11] reviews the integration of Machine Learning (ML) into Microcontroller Units (MCUs) via TinyML. They explore the available frameworks for integrating ML into MCUs and present a real use case of intelligent interface selection in multi-Radio Access Technology (RAT) devices. Through experimentation, the authors find that decision trees and Random Forest (RF) offer optimal performance on an Arduino Uno board.
12. Rakib et al. [12] introduces a lightweight CNN model for plant leaf disease classification, achieving 98% accuracy with int8 quantization, enabling deployment on microcontrollers like ESP32-CAM. The model's small size (28KB) and fast inference time (39.2ms) make it efficient and sustainable for real-time disease detection. This end-to-end solution offers practicality for farmers, potentially mitigating food shortages due to plant diseases. Future research aims to enhance the system's efficacy for recognizing additional diseases and potentially developing a mobile application for disease identification and remedy suggestions, expanding its accessibility and impact.

13. A. Paul et al. [13] presents an IoT-based education management system utilizing the ESP-32 CAM for face recognition and Flutter for mobile app development. Integrated with TinyML and Edge Impulse, it ensures accurate attendance tracking and efficient communication among stakeholders. The system's modular design and secure data storage in Google Sheets enhance scalability and durability. Through real-time image processing and cloud-based attendance recording, it streamlines education administration, offering features like class notifications and event updates. Overall, the system revolutionizes educational processes, improving organization, attendance, and communication, thereby enhancing the overall educational experience for students, professors, and alumni.
14. K. Dong et al. [14] compares MobileNetV1 and MobileNetV2 models on two datasets, highlighting MobileNetV2's superior accuracy and shorter training duration. Key contributions include performance analysis and visualization using saliency maps, affirming MobileNetV2's efficacy in image classification. Future research could benefit from expanding datasets and implementing derivative functions like segmentation and object detection to enhance MobileNetV2's performance further. Overall, MobileNetV2 emerges as a promising solution for image classification tasks, offering improved accuracy and efficiency compared to MobileNetV1.
15. Gulzar et al. [15] proposes a deep learning approach, TL - MobileNetV2, for fruit classification achieving 99% accuracy. The model is based on MobileNetV2 architecture with additional layers for improved efficiency. It outperforms previous methods and shows promise for practical deployment in mobile applications. Future work aims to expand the dataset and enhance the mobile app for broader fruit classification. Additionally, comparative analysis with other CNN models will be con-

ducted to identify the most effective approach. This research highlights the potential of deep learning in precision agriculture, specifically in fruit classification, quality analysis, and disease prediction, paving the way for enhanced agricultural practices.

16. Indraswari et al. [16] proposes a melanoma classification method using deep learning and transfer learning, leveraging MobileNetV2 for lightweight processing. The model achieves up to 85% accuracy on ISIC-Archive dataset, outperforming ResNet50V2, InceptionV3, and InceptionResNetV2. MobileNetV2's efficiency makes it suitable for deployment on mobile devices, aiding early melanoma detection. However, addressing imbalanced datasets remains a challenge. The proposed method shows promise for practical implementation, contributing to the early detection of melanoma and potentially reducing mortality rates associated with this common cancer.
17. Warden et al. [17] proposes the implementation of TensorFlow Lite on Arduino and ultra-low-power microcontrollers that enables efficient machine learning inference at the edge. By converting pre-trained models to TensorFlow Lite format and integrating them into Arduino IDE, developers can achieve real-time inference for sensor data processing and more. Optimizing for memory and computational efficiency is crucial, along with testing accuracy and latency. Deploying on ultra-low-power MCUs extends battery life, enabling standalone operation without cloud dependency. This approach enhances privacy and reduces latency, making it ideal for various IoT and embedded applications requiring intelligent edge computing.
18. Zim et al. [18]. This paper proposes that the integration of machine learning with microcontrollers like ESP32 opens new avenues for edge

computing. Utilizing the Xtensa LX6 microprocessor’s neural network capabilities, this study explores efficient neural network acceleration without specialized hardware. Through empirical analysis, it examines feedforward propagation speeds across one and two cores, considering Amdahl’s law for multicore processing efficiency. Results suggest potential speedup benefits in neural network execution, paving the way for optimized machine learning on resource-constrained devices. This research contributes insights into leveraging microcontroller capabilities for neural network applications, advancing edge computing solutions for diverse IoT deployments.

19. Mach et al. [19] explore the application of high-speed brushless DC (BLDC) motors in various industrial contexts, including Unmanned Aerial Vehicles (UAVs), emphasizing their compactness, efficiency, and torque advantages. It discusses the differences between out-runner and in-runner BLDC motors, highlighting the former’s suitability for UAVs due to their higher torque-to-speed ratio and absence of gearboxes. Furthermore, it details the hardware and software components of Electronic Speed Control (ESC) systems for BLDC motors, showcasing their integration with microcontrollers like Arduino for motor control. Experimental validation demonstrates the feasibility of using Arduino for BLDC motor control, setting the stage for further research in UAV applications.
20. Sudharsan et al. [20] introduce the TinyML-CAM pipeline, offering an end-to-end solution for designing and deploying customized machine learning-based image recognition systems on IoT and AIoT boards. It achieves ultra-lightweight performance, consuming only 1 kB on ESP32, with inference speeds of approximately 12 ms. This pipeline is the first to perform RF-based image recognition on AIoT boards and is open-source for reproducibility. Future work aims to further reduce pro-

cessing time to enhance frame rates, followed by comprehensive testing across various datasets, machine learning algorithms, and IoT boards, akin to the TinyML benchmark.

Chapter 3

Design and Hardware Components

3.1 Robot Layout Design

Robot chassis and structure should be compact and optimum for movement of robot and the transition between both cleaning mechanism should be flexible and easy for servo to pull off.

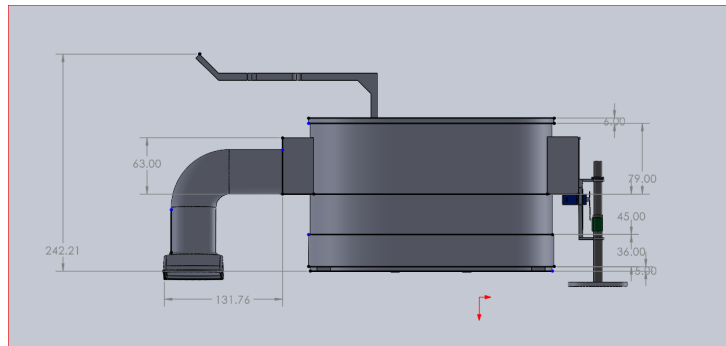


Figure 3.1: Basic Robot Layout

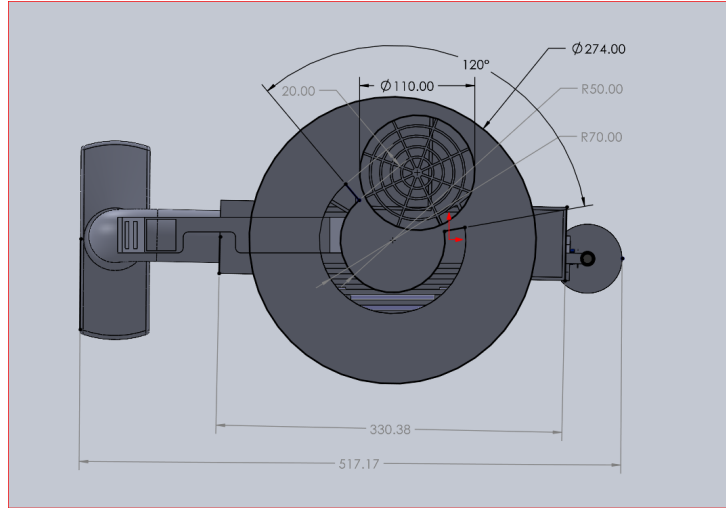


Figure 3.2: Top View of Robot

3.1.1 Chassis Layout

In our robot the outer layout is divided into two cylinders where one is mounted on the other. The lower cylinder is the part connected to the wheels, which are connected to gear motors and contains the processor and brain of the robot and other key parts like battery (power source) and storage unit inside of it and proximity sensors outside of it. And a vertical fixed fin-like section which is attached to one of the vital parts, camera module and ultra-sonic sensor, eyes of the robot. And the upper cylinder mainly consists of the complete cleaning mechanism and a cleavage for the fin-like section to pass. The two cylinders are connected through a circular cross-sectional partition which consists of a servo motor to rotate the upper cylinder and it also acts like a passage for wires or connections between sensors, actuators to the processor (Arduino MEGA). The upper cylinder space is primarily for accommodation of the solid debris collection bag and fan and other components required for vacuum suction. The outer shape of the cylinder is altered for the design of each cleaning mechanism.

3.1.2 Cleaning Mechanism

Advanced robotic systems in this section enable precise navigation, avoiding obstacles while covering designated areas for comprehensive cleaning. This section plays a vital role in the robot's choosing what type of cleaning mechanism should work. The upper part of robot turns 180 degrees for placing the desired type of mechanism forward with equal characteristic length between debris and reference point, which is camera point.

3.1.2.1 Vacuum Section

The vacuum section of an autonomous cleaning robot incorporates mechanisms akin to traditional vacuum cleaners but tailored for robotic operation. It comprises a suction mechanism, typically driven by a motor, to pull in air and debris from surfaces. This section also integrates filters to trap particles while allowing clean air passage, contributing to improved air quality. Debris collected is stored in a dedicated compartment within the robot as shown in fig 3.3. Advanced sensors and navigation systems enable the robot's vacuum section to intelligently detect and navigate around obstacles, adjusting suction power based on surface types for effective cleaning. This section's design ensures efficient and autonomous vacuuming, a critical function in the robot's cleaning process.

3.1.2.2 Mopping Section

The mopping section in an autonomous cleaning robot focuses on efficiently and autonomously cleaning surfaces through specialized mechanisms. Here, a mopping pad or cloth, either reusable or disposable, is utilized to scrub and wipe the surface, removing stains, spills, or residue.

3.1.3 Camera Placement

The placement of eyes of the robot is most crucial part of the project. Because it decides the height,length and other dimensions of the debris, on which the

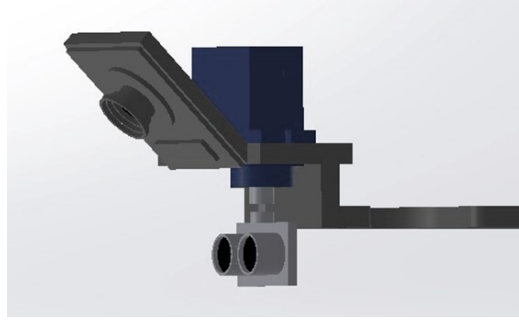


Figure 3.3: Camera Module Placement

travelling path and cleaning length depends. So We chose optimum way to place them. The camera is connected to servo motor which rotates to detect the debris first, so the robot moves first to the location where fixed camera takes photos which is classified through machine learning.

3.2 Components Required

3.2.1 Arduino Mega

The Arduino Mega, with its expanded input/output capacity, robust memory, and processing capabilities, plays a pivotal role in facilitating machine learning. While not traditionally a platform for complex machine learning models due to memory and computational limitations, the Mega's ample resources allow for simpler machine learning algorithms or decision-making logic. This capability enables the robot to learn from sensor data, make adaptive decisions, and enhance its cleaning strategies based on environmental feedback. While not as powerful as dedicated machine learning platforms, the Mega's computational strength supports basic learning processes crucial for autonomy and adaptive behavior in cleaning routines. Despite not being the most powerful microcontroller available, its balance between features and cost renders it a cost-effective choice for projects with specific needs. Overall, the Arduino Mega's adaptability, scalability, and compatibility with numerous components solidify its position as a cornerstone for a broad spectrum

in this project.

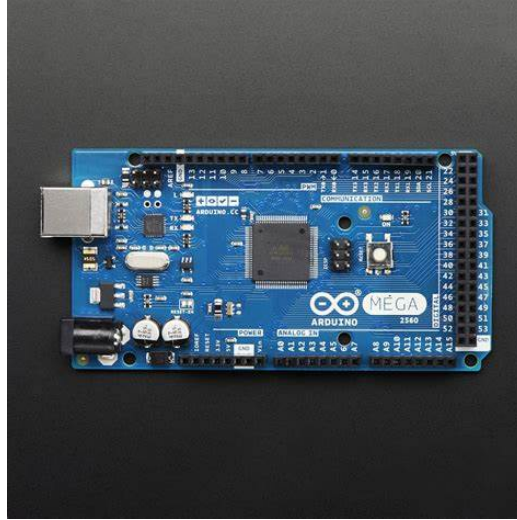


Figure 3.4: Arduino Mega 2560

3.2.2 Motor Driver and Motor Shield

In Arduino projects, motor drivers and motor shields are essential for motor control. A motor driver, like the L298N or L293D, acts as an intermediary between the microcontroller and the motors, amplifying low-current control signals from the Arduino to higher current levels needed by the motors, and providing functionalities like current and voltage regulation, direction control, and speed control via PWM. Motor shields, such as the Arduino Motor Shield Rev3, are expansion boards that stack directly onto the Arduino, simplifying wiring and allowing for easy control of multiple motors. They integrate multiple motor drivers and additional components for enhanced operation and protection, often supporting up to four DC motors or two stepper motors. Motor shields are ideal for complex projects requiring multi-motor control and offer additional features like current sensing and braking capabilities, making them user-friendly and suitable for both beginners and advanced users.

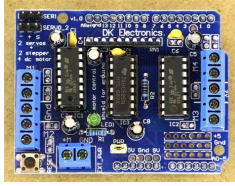


Figure 3.5: L293D Motor Shield

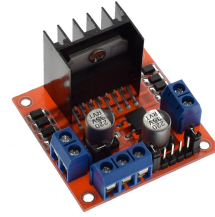


Figure 3.6: L298N Motor Driver

3.2.3 BLDC Motor and ESC

A BLDC (Brushless DC) motor is a type of electric motor that operates using electronic commutation instead of brushes and a commutator. It requires an ESC (Electronic Speed Controller) to control its speed and direction by varying the voltage and frequency of the electrical pulses sent to the motor windings. The ESC interprets commands from a controller or microcontroller and adjusts the power supply to the BLDC motor accordingly, enabling precise and efficient motor control in applications such as drones, RC vehicles, and industrial automation.



Figure 3.7: BLDC Motor



Figure 3.8: Electronic Speed Controller

3.2.4 Servomotor

A servo motor is a rotary actuator that allows precise control of angular position. It consists of a motor, a gearbox, and a feedback control system. Servo motors are commonly used in robotics, RC vehicles, and automation systems for their accuracy, reliability, and ease of control.



Figure 3.9: SG90



Figure 3.10: MG985

3.2.5 ESP 32 Camera module

The ESP32 camera module integrates an OV2640 camera sensor with an ESP32 microcontroller, providing Wi-Fi and Bluetooth connectivity along with the capability to capture and process images. It features a compact form factor and supports various image resolutions and formats. The module enables developers to easily create IoT projects, surveillance systems, and video streaming applications. With its onboard processing power and wireless connectivity, the ESP32 camera module offers a versatile solution for visual data acquisition and transmission in embedded systems.

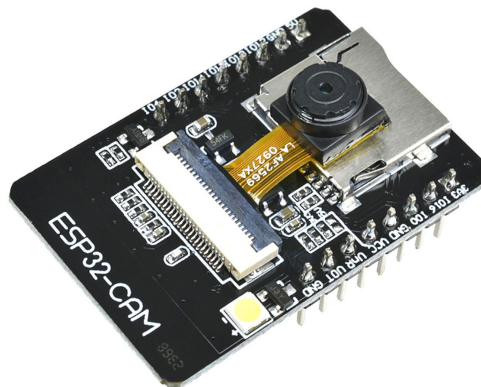


Figure 3.11: ESP 32 Camera Module

3.2.6 Power Supply

9V Battery is common for powering small electronic devices, providing a convenient and portable power source. A 3.6V rechargeable battery is often

used in portable electronics, offering a compact and reliable power solution.



Figure 3.12: 9V Battery



Figure 3.13: Li-ion Battery

S.NO	Component	Quantity
1.	Arduino Mega 2560	1
2.	L293 Motor Driver Shield	1
3.	L298 Motor Driver	1
4.	Camera	1
5.	SD Card Module	1
6.	SD card	1
7.	IR Sensor	1
8.	BLDC Motor	1
9.	ESC	1
10.	Gear Motor with Wheels	4
11.	DC Motor	1
12.	LI-ion Rechargeable Batteries	6
13.	9V Batteries	2
14.	Battery Holder(1*3)	2

Table 3.1: List of Components

Chapter 4

Machine Learning Model

In this chapter, we present the development and optimization of a machine learning model for debris classification using TensorFlow and Keras. The model is designed to classify images of debris into two categories: "solid" and "liquid".

4.1 Dataset Preparation

The dataset used for training, validation, and testing was collected from various sources and manually labeled into the two classes: "solid" and "liquid". The dataset was split into training, validation, and testing sets with a ratio of 70%, 15%, and 15% respectively. Additionally, data augmentation techniques such as rotation, scaling, and flipping were applied to increase the diversity of the training dataset and improve model generalization.

4.2 Model Architecture

We utilized transfer learning to leverage pre-trained convolutional neural network (CNN) architectures for feature extraction. Specifically, we employed the MobileNetV2 architecture as the base model due to its efficiency and effectiveness in image classification tasks. The MobileNetV2 model was ini-

tialized with ImageNet weights and fine-tuned on the debris classification dataset.

4.3 Training Process

The training process involved optimizing the model's parameters to minimize the categorical cross-entropy loss function. We used the Adam optimizer with a learning rate of 0.001 and early stopping to prevent overfitting. The model was trained for 15 epochs, with batch sizes of 10 images per iteration.

4.3.1 Model Preparation

```
import tensorflow as tf
import numpy as np
from tensorflow import keras
from tensorflow.keras.layers import Dense # type: ignore
from tensorflow.keras.optimizers import Adam # type: ignore
from tensorflow.keras.preprocessing.image import ImageDataGenerator # type: ignore
from sklearn.metrics import confusion_matrix # type: ignore
import os
import shutil
import random
import glob
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
from plot_confusionMatrix import pcm
os.chdir('F:\ml\major project\debris_data')
if os.path.isdir('train/solid') is False:
    os.makedirs('train/solid')
    os.makedirs('train/liquid')
    os.makedirs('valid/solid')
    os.makedirs('valid/liquid')
    os.makedirs('test/solid')
    os.makedirs('test/liquid')
    for i in random.sample(glob.glob('solid*'),100):
        shutil.move(i,'train/solid' )
    for i in random.sample(glob.glob('liquid*'),100):
        shutil.move(i,'train/liquid' )
    for i in random.sample(glob.glob('solid*'),40):
        shutil.move(i,'valid/solid' )
    for i in random.sample(glob.glob('liquid*'),40):
        shutil.move(i,'valid/liquid' )
    for i in random.sample(glob.glob('solid*'),20):
        shutil.move(i,'test/solid' )
    for i in random.sample(glob.glob('liquid*'),20):
        shutil.move(i,'test/liquid' )
os.chdir('.../.../')
```

```

train_path=r'F:\ml\major project\debris_data\train'
valid_path=r'F:\ml\major project\debris_data\valid'
test_path=r'F:\ml\major project\debris_data\test'
train_batches=ImageDataGenerator(preprocessing_function=tf.keras.applications.vgg16.preprocess_input)\
| .flow_from_directory(directory=train_path,target_size=(224,224), classes=['solid','liquid'], batch_size=10)
valid_batches=ImageDataGenerator(preprocessing_function=tf.keras.applications.vgg16.preprocess_input)\
| .flow_from_directory(directory=valid_path,target_size=(224,224), classes=['solid','liquid'], batch_size=10)
test_batches=ImageDataGenerator(preprocessing_function=tf.keras.applications.vgg16.preprocess_input)\
| .flow_from_directory(directory=test_path,target_size=(224,224), classes=['solid','liquid'], batch_size=10, shuffle=False)

imgs, labels= next(train_batches)
base_model = keras.applications.MobileNetV2(include_top=False, weights='imagenet')
# Freeze all layers except the last few
for layer in base_model.layers[:-5]:
    layer.trainable = False
from tensorflow.keras.layers import GlobalAveragePooling2D # type: ignore
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
last = Dense(2, activation='softmax')(x)

model = keras.Model(inputs=base_model.input, outputs=last)

model.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model with early stopping
early_stopping = keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True)

history = model.fit(train_batches, validation_data=valid_batches, epochs=15, callbacks=[early_stopping], verbose=2)
# Evaluate on test data
test_loss, test_acc = model.evaluate(test_batches)
print('Test accuracy:', test_acc)

# Save the entire model
model.save('mp_model.h5')

```

```

predictions = model.predict(x=test_batches, verbose=1)
cnn_pred=np.round(predictions) # its 100*2 array, and output like [1. 0.] [1. 0.] [0. 1.]

# Get the actual labels for the test batch
actual_labels = test_batches.classes
print(type(actual_labels))
# Convert the predicted probabilities to class labels (0 or 1)
predicted_labels = np.argmax(cnn_pred, axis=-1)
print(type(predicted_labels))
# Compare the predicted labels with the actual labels
correct_predictions = np.equal(predicted_labels, actual_labels)
print(correct_predictions)
# Calculate the accuracy
accuracy = np.mean(correct_predictions)
print("Accuracy:", accuracy)

from plot_confusionMatrix import pcm

cm=confusion_matrix(y_true=test_batches.classes, y_pred=np.argmax(predictions, axis=-1))
test_batches.class_indices
cm_plot_labels=['liquid','solid']
pcm(cm=cm, classes= cm_plot_labels,title='confusion Matrix')

```

Figure 4.1: CNN Model Code in VSCODE

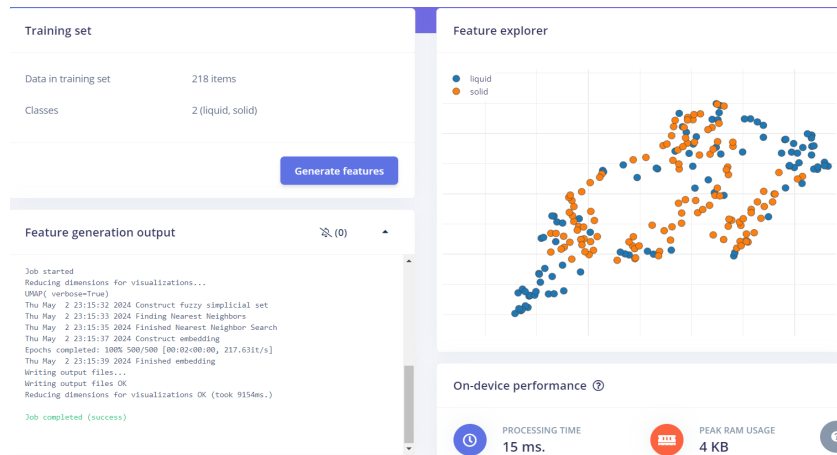


Figure 4.2: Features Map

4.3.2 Accuracy Results

4.3.2.1 Training Results with Edge Impulse Plotter



Figure 4.3: Training Results with Edge Impulse Plotter

4.3.2.2 Testing Results with Edge Impulse Plotter



Figure 4.4: Testing Results with Edge Impulse Plotter

```
Epoch 1/15
20/20 - 14s - loss: 0.5536 - accuracy: 0.7850 - val_loss: 0.3556 - val_accuracy: 0.9000
Epoch 2/15
20/20 - 8s - loss: 0.1487 - accuracy: 0.9500 - val_loss: 0.3497 - val_accuracy: 0.9000
Epoch 3/15
20/20 - 8s - loss: 0.0431 - accuracy: 0.9850 - val_loss: 1.1639 - val_accuracy: 0.8250
Epoch 4/15
20/20 - 8s - loss: 0.0258 - accuracy: 0.9950 - val_loss: 0.5398 - val_accuracy: 0.9000
Epoch 5/15
20/20 - 9s - loss: 0.0324 - accuracy: 0.9850 - val_loss: 0.3576 - val_accuracy: 0.9125
4/4 [=====] - 1s 339ms/step - loss: 0.1210 - accuracy: 0.9500
Test accuracy: 0.949999988079071
C:\Users\DELL\anaconda3\envs\tensor\lib\site-packages\keras\utils\generic_utils.py:497: Cu
ayers require a config and must override get_config. When loading, the custom mask layer m
jects argument.
category=CustomMaskWarning)
4/4 [=====] - 2s 288ms/step
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
[ True True True True False True True True True True True True
 True True True True True True True True True True True True
 True True True True True False True True True True True True
 True True True True]
Accuracy: 0.95
Confusion matrix, without normalization
[[19  1]
 [ 1 19]]
```

Figure 4.5: Terminal Result with Confusion Matrix

4.4 Evaluation Metrics

To evaluate the performance of the trained model, we used accuracy as the primary metric, calculated as the ratio of correctly classified images to the total number of images in the test set. Additionally, we computed the confusion matrix to analyze the model’s performance in classifying ”solid” and ”liquid” debris.

4.5 Model Optimization

To optimize the model for deployment on resource-constrained devices, we employed two optimization techniques: pruning and quantization. Pruning involves removing unnecessary connections in the neural network to reduce model size and computational complexity. We used TensorFlow Model Optimization Toolkit to apply magnitude-based weight pruning to the trained model.

4.6 Quantization-Aware Training

Quantization-aware training is a technique used to train models for deployment on low-power devices by simulating the effects of reduced precision arithmetic during training. We applied quantization-aware training to the pruned model, which involves training the model with reduced precision (e.g., 8-bit integers) while minimizing accuracy loss.

4.7 TensorFlow Lite Conversion

Finally, we converted the quantized and pruned model into TensorFlow Lite format for deployment on mobile and embedded devices. TensorFlow Lite provides a lightweight runtime for executing machine learning models on edge devices with limited computational resources.

4.8 Code Implementation for Pruning

Below is the Python code implementation for model pruning, quantization-aware training, and TensorFlow Lite conversion:

```
import tensorflow as tf
from tensorflow.keras.models import load_model # type: ignore
import tensorflow_model_optimization as tfmot # type: ignore
import numpy as np
from tensorflow.keras.preprocessing.image import ImageDataGenerator # type: ignore

# Load your Keras model
model = load_model(r'F:\ml\mp_model_squeezenet.h5')

# Apply pruning to the model
prune_low_magnitude = tfmot.sparsity.keras.prune_low_magnitude
pruning_params = {
    'pruning_schedule': tfmot.sparsity.keras.PolynomialDecay(initial_sparsity=0.50,
                                                              final_sparsity=0.90,
                                                              begin_step=0,
                                                              end_step=1000)
}
pruned_model = prune_low_magnitude(model, **pruning_params)

# Compile the pruned model
pruned_model.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])

# Remove the PruneLowMagnitude wrappers
model_for_export = tfmot.sparsity.keras.strip_pruning(pruned_model)
# Quantize the pruned model
quantize_model = tfmot.quantization.keras.quantize_model

# Quantize the model
q_aware_pruned_model = quantize_model(model_for_export)

# Compile the quantized model
q_aware_pruned_model.compile(optimizer='adam',
                            loss='sparse_categorical_crossentropy',
                            metrics=['accuracy'])
```

```

#q_aware_pruned_model.summary()

converter = tf.lite.TFLiteConverter.from_keras_model(q_aware_pruned_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]

data_generator = ImageDataGenerator(rescale=1./255)

dataset_dir = r"F:\ml\major project\opt"

# Create data generators for the representative dataset
representative_batches = data_generator.flow_from_directory(
    dataset_dir,
    target_size=(227, 227), # Adjust the target size based on your model's input shape
    batch_size=1, # Use batch size 1 for representative dataset
    class_mode=None, # No need for labels in the representative dataset
    shuffle=False # Ensure consistent order of images
)

# Define the representative dataset generator function
def representative_dataset_gen():
    # Keep yielding batches until the generator is exhausted
    while True:
        batch = representative_batches.next()
        yield [batch.astype(np.float32)]

converter.representative_dataset = representative_dataset_gen
# Convert the model
tflite_qaware_pruned_model = converter.convert()
# Write the converted model to a file
tflite_file = "tflite_qaware_pruned_model.tflite"
with open(tflite_file, 'wb') as f:
    f.write(tflite_qaware_pruned_model)
print(f"Quantized and pruned TensorFlow Lite model written to: {tflite_file}")

```

Figure 4.6: Data Quantisation Code

4.8.1 Testing

```

import tensorflow as tf
from keras.preprocessing.image import load_img, img_to_array
from time import time
import matplotlib.pyplot as plt
import os
from tensorflow.keras.applications.vgg16 import preprocess_input # type: ignore
image=load_img(r'F:\ml\jpg2png\solid_10.png',target_size=(224,224))
image=img_to_array(image)
image=image.reshape((1,image.shape[0],image.shape[1],image.shape[2]))
print(image.shape)
image= preprocess_input(image)

model_path1 = r"F:\ml\major project\code\tflite_qaware_pruned_model.tflite"
interpreter=tf.lite.interpreter(model_path=model_path1)

input_details=interpreter.get_input_details()
output_details=interpreter.get_output_details()

# Set the input tensor
interpreter.set_tensor(input_details[0]['index'], image)

# Run inference
interpreter.invoke()

# Get the output tensor
output_data = interpreter.get_tensor(output_details[0]['index'])
print("Predicted class:", output_data)

```

Figure 4.7: Testing with Image from System

4.8.2 Quantized Model



Figure 4.8: Training Performance of Quantized Model(int8)

4.8.3 Results

```
2024-05-16 09:00:18.970388: I tensorflow/stream_executor/cuda/cuda_diagnostics.cc:169] retrieving CUDA
tion for host: DESKTOP-0ERHONG
2024-05-16 09:00:18.971124: I tensorflow/stream_executor/cuda/cuda_diagnostics.cc:176] hostname: DESKTO
2024-05-16 09:00:18.971682: I tensorflow/core/platform/cpu_feature_guard.cc:142] This TensorFlow binary
oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-criti
VX AVX2
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
2024-05-16 09:00:20.960196: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:185] None of the
Passes are enabled (registered 2)
Predicted class: solid
```

Figure 4.9: Result from Previous Code

4.9 Integration with Edge Impulse

Following the optimization steps, we proceeded to integrate our TensorFlow Lite model with Edge Impulse, a platform specializing in deploying machine learning models on edge devices. This integration facilitated further optimization and deployment on resource-constrained devices like the ESP32 CAM.

4.10 Further Optimization and Quantization

Upon integration with Edge Impulse, we conducted additional optimization and quantization steps to tailor the model specifically for deployment on the ESP32 CAM. These optimizations were crucial to ensure optimal performance within the constraints of the target hardware.

4.11 Library Download for ESP32 CAM

After completing the optimization process on Edge Impulse, we downloaded the optimized model as a library tailored for the ESP32 CAM. This library contained all the necessary components and configurations to seamlessly deploy the machine learning model on the ESP32 CAM device.

Chapter 5

Arduino Software and Robot Assembly

5.1 Hardware Configuration

The robot's hardware setup comprises an Arduino Mega microcontroller, various sensors, actuators, and motor drivers. The key components include

Arduino Mega: Used as the main control unit for sensor data processing and actuator control.

Ultrasonic Sensor: Utilized for obstacle detection and path planning. Servo Motors: Employed for controlling the orientation of the cleaning apparatus and sensor array.

Brushless DC (BLDC) Motor: Used for locomotion and maneuvering. Motor Drivers: Required for interfacing the motors with the Arduino Mega.

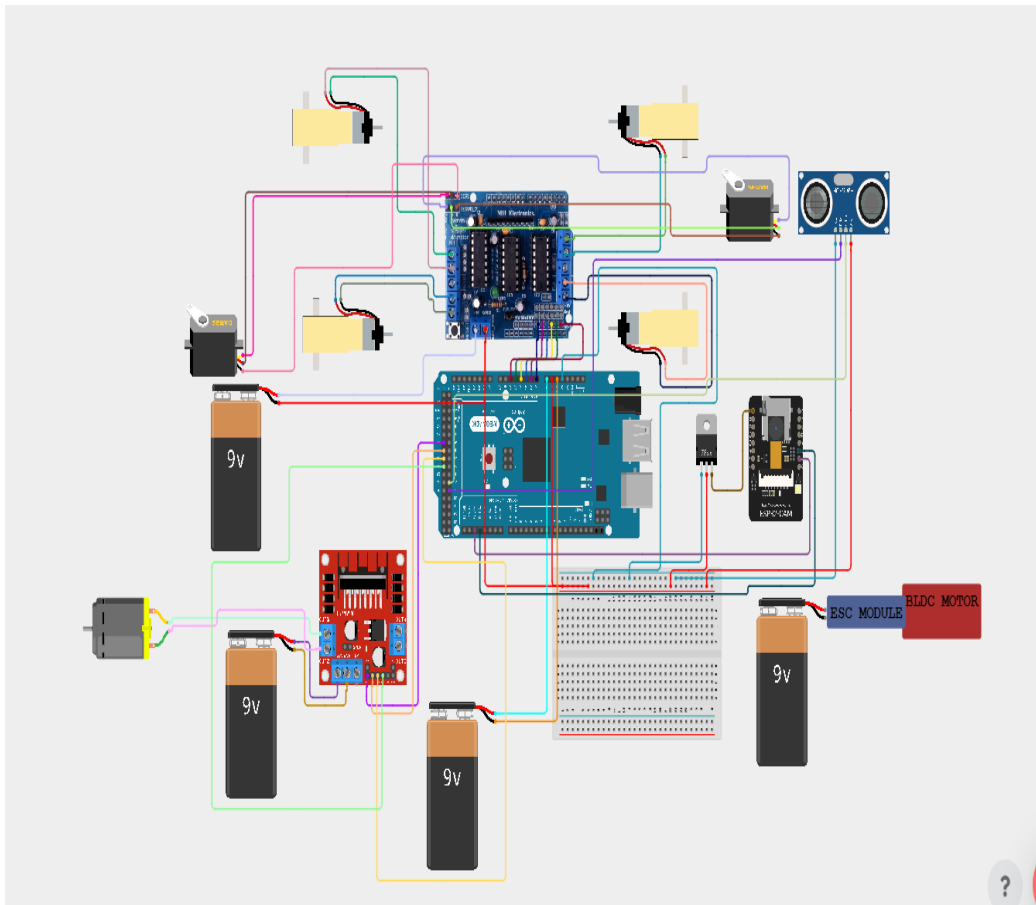


Figure 5.1: Arduino Circuit

5.2 Arduino Software Architecture

The Arduino software architecture is designed to facilitate efficient sensor data processing, motor control, and communication with external devices. The key components and their functionalities include:

5.2.1 Sensor Interface

Ultrasonic Sensor Interface: Implemented to measure distances and detect obstacles in the robot's path.

Camera Module Interface: Established to receive classification labels from the ESP32-CAM module.

5.2.2 Actuator Control

Servo Motor Control: Implemented to adjust the orientation of the cleaning apparatus and sensor array.

BLDC Motor Control: Facilitated to control the robot's locomotion and maneuvering.

5.2.3 Communication

Serial Communication: Utilized for communication with the ESP32-CAM module and exchange of classification labels.

Inter-Arduino Communication: Established between the Arduino Mega and other Arduino-compatible boards for coordinated operation.

5.3 Arduino Code Implementation

5.3.1 Arduino Mega


```

1  #define BUFFER_SIZE 20
2  #include <Servo.h>
3  #include <AFMotor.h>
4  #include <NewPing.h>
5
6  // Define the baud rate for serial communication
7  #define SERIAL_BAUD_RATE 115200
8  // Create Servo objects
9  Servo servo1;
10 Servo servo2;
11 #define BLDC_SPEED_PIN 24
12 #define MOP_MOTOR_PIN 22
13
14 #define BLDC_RUN_DURATION 15000 // 15 seconds
15 #define MOP_RUN_DURATION 15000 // 15 seconds for mop motor
16 // Define ultrasonic sensor pins
17 #define TRIGGER_PIN 7
18 #define ECHO_PIN 6
19 #define MAX_DISTANCE 200
20
21 AF_DCMotor motor1(1, MOTOR12_1KHZ);
22 AF_DCMotor motor2(2, MOTOR12_1KHZ);
23 AF_DCMotor motor3(3, MOTOR34_1KHZ);
24 AF_DCMotor motor4(4, MOTOR34_1KHZ);
25
26 NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
27
28 int classificationLabel = 0;
29
30 void setup() {
31     // Initialize serial communication
32     Serial.begin(SERIAL_BAUD_RATE);
33     servo1.attach(10); // You may need to adjust this pin number
34     servo2.attach(11); // You may need to adjust this pin number
35     pinMode(MOP_MOTOR_PIN, OUTPUT);
36 }

```

```

38 void loop() {
39     int directionAngle = random(-90, 91);
40     servo1.write(90 + directionAngle);
41     delay(1000); // Delay to allow the servo1 to settle
42     if (isPathClear(directionAngle, 50)) {
43         moveInDirection(directionAngle, 30);
44         delay(5000); // Delay after moving
45
46         moveInDirection(directionAngle, 10);
47         delay(1000); // Delay after moving 10cm
48     if (Serial.available()) {
49         // Read the incoming data
50         classificationLabel = Serial.readString().toInt();
51         // Clear the serial buffer
52         while (Serial.available()) {
53             Serial.read();
54         }
55     }
56
57     if (classificationLabel == 1) {
58         activateVacuum(); // Call the function to activate
59     } else if (classificationLabel == 0) {
60         activateMop(); // Call the function to activate th
61     }
62 }
63 unsigned long startTime = millis();
64 while (millis() - startTime < 20000) {
65     // Wait for 30 seconds
66     if (Serial.available() > 0) {
67         // Read the incoming byte from ESP32-CAM
68         char receivedChar = Serial.read();
69         if (receivedChar == '1') {
70             classificationLabel = 1;
71         } else if (receivedChar == '0') {
72             classificationLabel = 0;

```

```

74     }
75 }
76 }
77 void moveStop() {
78     motor1.run(RELEASE);
79     motor2.run(RELEASE);
80     motor3.run(RELEASE);
81     motor4.run(RELEASE);
82 }
83
84 void moveForward() {
85     motor1.run(FORWARD);
86     motor2.run(FORWARD);
87     motor3.run(FORWARD);
88     motor4.run(FORWARD);
89 }
90
91 void moveBackward() {
92     motor1.run(BACKWARD);
93     motor2.run(BACKWARD);
94     motor3.run(BACKWARD);
95     motor4.run(BACKWARD);
96 }
97
98 void moveInDirection(int angle, int distance_cm) {
99     if (angle < 0) {
100         // Turn right
101         motor1.run(FORWARD);
102         motor2.run(FORWARD);
103         motor3.run(BACKWARD);
104         motor4.run(BACKWARD);
105     } else {
106         // Turn left
107         motor1.run(BACKWARD);
108         motor2.run(BACKWARD);

```

Figure 5.2: Arduino Mega Code

5.3.2 ESP 32 code

```
1  #include <debris_classify_inferencing.h>
2  #include "edge-impulse-sdk/dsp/image/image.hpp"
3  #include "esp_camera.h"
4
5  // Define camera model
6  #define CAMERA_MODEL_AI_THINKER // Change this according to your camera model
7
8  // Define camera pins based on the selected model
9  #if defined(CAMERA_MODEL_AI_THINKER)
10 #define PWDN_GPIO_NUM    32
11 #define XCLK_GPIO_NUM    0
12 #define SIOD_GPIO_NUM    26
13 #define SIOC_GPIO_NUM    27
14 #define Y9_GPIO_NUM      35
15 #define Y8_GPIO_NUM      34
16 #define Y7_GPIO_NUM      39
17 #define Y6_GPIO_NUM      36
18 #define Y5_GPIO_NUM      21
19 #define Y4_GPIO_NUM      19
20 #define Y3_GPIO_NUM      18
21 #define Y2_GPIO_NUM       5
22 #define VSYNC_GPIO_NUM   25
23 #define HREF_GPIO_NUM    23
24 #define PCLK_GPIO_NUM    22
25 #endif
26
27 #define EI_CAMERA_RAW_FRAME_BUFFER_COLS    320
28 #define EI_CAMERA_RAW_FRAME_BUFFER_ROWS    240
29 #define EI_CAMERA_FRAME_BYTE_SIZE         3
30
31 #define CAPTURE_INTERVAL_MS    30000
32
33 // Define serial baud rate
34 #define SERIAL_BAUD_RATE    115200
35
36 // Define classification label types
37 enum ClassType {
38     SOLID,
39     LIQUID
40 };
41
```

```

44 static uint8_t *snapshot_buf;
45 static HardwareSerial Serial2(2); // Using Serial2 for communication with Arduino Mega
46
47 // Camera configuration
48 static camera_config_t camera_config = {
49     .pin_pwdn = PWDN_GPIO_NUM,
50     .pin_xclk = XCLK_GPIO_NUM,
51     .pin_sscb_sda = SIOD_GPIO_NUM,
52     .pin_sscb_scl = SIOC_GPIO_NUM,
53     .pin_d7 = Y9_GPIO_NUM,
54     .pin_d6 = Y8_GPIO_NUM,
55     .pin_d5 = Y7_GPIO_NUM,
56     .pin_d4 = Y6_GPIO_NUM,
57     .pin_d3 = Y5_GPIO_NUM,
58     .pin_d2 = Y4_GPIO_NUM,
59     .pin_d1 = Y3_GPIO_NUM,
60     .pin_d0 = Y2_GPIO_NUM,
61     .pin_vsync = VSYNC_GPIO_NUM,
62     .pin_href = HREF_GPIO_NUM,
63     .pin_pclk = PCLK_GPIO_NUM,
64     .xclk_freq_hz = 20000000,
65     .ledc_timer = LEDC_TIMER_0,
66     .ledc_channel = LEDC_CHANNEL_0,
67     .pixel_format = PIXFORMAT_JPEG,
68     .frame_size = FRAMESIZE_QVGA,
69     .jpeg_quality = 12,
70     .fb_count = 1,
71     .fb_location = CAMERA_FB_IN_PSRAM,
72     .grab_mode = CAMERA_GRAB_WHEN_EMPTY,
73 };
74
75 bool ei_camera_init();
76 void ei_camera_deinit();
77 bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf);
78 ClassType run_inference();
79 void send_classification(ClassType class_type);
80
81 void setup() {
82     Serial.begin(SERIAL_BAUD_RATE);
83     Serial2.begin(SERIAL_BAUD_RATE); // Initialize Serial2 for communication with Arduino Mega
84

```

```

85 while (!Serial); // Wait until Serial connection is established
86 Serial.println("Edge Impulse Inferencing Demo");
87
88 if (ei_camera_init() == false) {
89     Serial.println("Failed to initialize Camera!");
90 } else {
91     Serial.println("Camera initialized");
92 }
93 }
94
95 void loop() {
96     delay(CAPTURE_INTERVAL_MS); // Wait for the capture interval
97
98     snapshot_buf = (uint8_t*)malloc(EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * EI_CAMERA_FRAME_BYTE_SIZE);
99     if (snapshot_buf == nullptr) {
100         Serial.println("ERR: Failed to allocate snapshot buffer!");
101         return;
102     }
103
104     // Capture an image
105     if (ei_camera_capture((size_t)EI_CLASSIFIER_INPUT_WIDTH, (size_t)EI_CLASSIFIER_INPUT_HEIGHT, snapshot_buf) == false) {
106         Serial.println("Failed to capture image");
107         free(snapshot_buf);
108         return;
109     }
110
111     // Run inference on the captured image
112     ClassType class_type = run_inference();
113
114     // Send the classification to Arduino Mega
115     send_classification(class_type);
116
117     free(snapshot_buf);
118 }
119
120 bool ei_camera_init() {
121     if (is_initialised) return true;
122
123     esp_err_t err = esp_camera_init(&camera_config);
124     if (err != ESP_OK) {
125         Serial.printf("Camera init failed with error 0x%x\n", err);

```

```

126     return false;
127 }
128
129 sensor_t *s = esp_camera_sensor_get();
130 if (s->id.PID == OV3660_PID) {
131     s->set_vflip(s, 1); // Flip the image vertically
132 }
133
134 is_initialised = true;
135 return true;
136 }
137
138 void ei_camera_deinit() {
139     esp_err_t err = esp_camera_deinit();
140     if (err != ESP_OK) {
141         Serial.println("Camera deinit failed");
142         return;
143     }
144
145     is_initialised = false;
146 }
147
148 bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) {
149     camera_fb_t *fb = esp_camera_fb_get();
150     if (!fb) {
151         Serial.println("Camera capture failed");
152         return false;
153     }
154
155     bool converted = fmt2rgb888(fb->buf, fb->len, PIXFORMAT_JPEG, out_buf);
156     esp_camera_fb_return(fb);
157
158     if (!converted) {
159         Serial.println("Conversion failed");
160         return false;
161     }
162
163     return true;
164 }
165
166 ClassType run_inference() {

```



```

166 ClassType run_inference() {
167     // Create signal structure for inference
168     ei::signal_t signal;
169     signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
170     signal.get_data = &ei_camera_get_data;
171
172     // Run the classifier
173     ei_impulse_result_t result = { 0 };
174     EI_IMPULSE_ERROR err = run_classifier(&signal, &result, false);
175     if (err != EI_IMPULSE_OK) {
176         Serial.printf("ERR: Failed to run classifier (%d)\n", err);
177         return LIQUID; // Default to LIQUID classification in case of error
178     }
179
180     if (result.classification[0].value > result.classification[1].value) {
181         return SOLID;
182     } else {
183         return LIQUID;
184     }
185 }
186
187 void send_classification(ClassType class_type) {
188     // Send the classification to Arduino Mega
189     if (class_type == SOLID) {
190         Serial2.println("Solid");
191     } else {
192         Serial2.println("Liquid");
193     }
194 }

```

Figure 5.3: ESP 32 Code

5.4 Robot Assembly and Integration

The robot assembly involves integrating the Arduino Mega, sensors, motors, and other components into a cohesive unit. The assembly process includes:

Mounting Components: Securely mounting the Arduino Mega, sensors, and actuators onto the robot chassis.

Wiring and Connections: Establishing electrical connections between the components and ensuring proper communication and power supply.

Calibration and Testing: Calibrating sensor readings, motor movements, and communication interfaces for optimal performance.

Integration with ESP32-CAM Module: Ensuring seamless communication and synchronization between the Arduino Mega and ESP32-CAM module for coordinated operation.

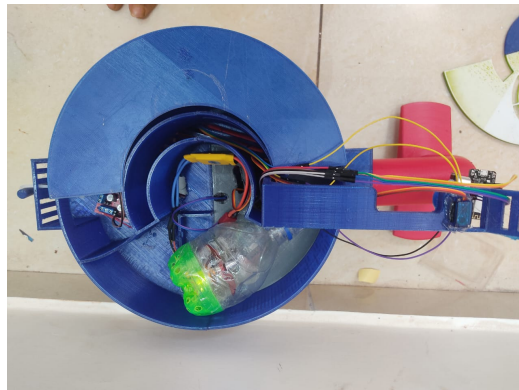


Figure 5.4: Top View of Robot and BLDC Motor with Vacuum Chamber

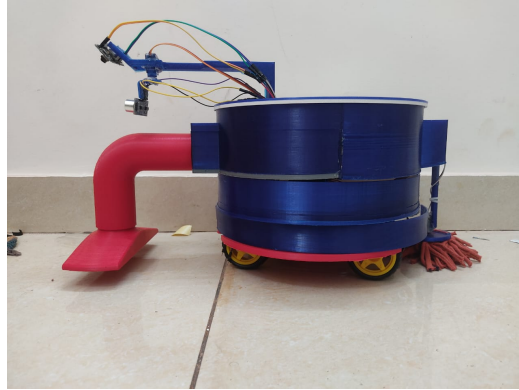


Figure 5.5: Side View of Robot with Suction Pipe



Figure 5.6: Top View of Robot with Camera Module

Chapter 6

Conclusion

Our journey began with the meticulous collection and preparation of a diverse dataset containing images of debris, categorized into "solid" and "liquid." This dataset served as the foundation for training our machine learning model. Through rigorous labeling and augmentation techniques, we ensured a balanced representation of debris types and enhanced the dataset's diversity to improve model generalization.

With the dataset ready, we turned to transfer learning, leveraging the MobileNetV2 architecture pre-trained on ImageNet. Fine-tuning this pre-trained model on our debris dataset, we tailored it to our specific classification needs. Transfer learning expedited model development while ensuring high performance, thanks to the inherent features learned from ImageNet.

Optimizing the model for deployment on resource-constrained edge devices was paramount. We adopted quantization-aware training (QAT), a technique simulating reduced-precision arithmetic during training. This approach significantly reduced the model's size without compromising accuracy, making it suitable for real-time inference on edge devices like the ESP32 CAM.

Integration with the Edge Impulse platform facilitated seamless deployment of our optimized model onto the ESP32 CAM using Arduino. This enabled real-time inference directly on the device, opening avenues for practi-

cal applications such as environmental monitoring and disaster management. Rigorous testing and validation procedures confirmed the model's robustness and accuracy across various debris types and environmental conditions.

While our project has achieved significant milestones, there are areas for further exploration and enhancement. Future work could involve refining the model architecture, exploring additional optimization techniques, and scaling the application's capabilities to address broader environmental challenges. Additionally, integrating real-time sensor data and advanced anomaly detection algorithms could augment the system's capabilities in monitoring and disaster response scenarios.

Chapter 7

References

- [1] Mbunge, Elliot, et al., Application of deep learning and machine learning models to detect COVID-19 face masks: a review, *Sustainable Operations and Computers*, 2 (2021): 235-245.
- [2] Zhao, Zhong-Qiu, et al., Object detection with deep learning: a review, *IEEE transactions on neural networks and learning systems*, 30.11 (2019): 3212-3232.
- [3] Ferentinos, Konstantinos P., Deep learning models for plant disease detection and diagnosis, *Computers and electronics in agriculture*, 145 (2018): 311-318.
- [4] Zech, John R., et al., Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: a cross-sectional study, *PLoS medicine*, 15.11 (2018): e1002683.
- [5] Sankar, M., Mudgal, D., Jalinder, M.M., et al, Green leaf disease detection using raspberry pi, *International Conference on Innovations in Information and Communication Technology (ICIICT)*, pp. 1-6. IEEE (2019).
- [6] Venu, D. N., IOT Surveillance Robot Using ESP-32 Wi-Fi CAM & Arduino, *International Journal of Food and Nutritional Sciences* 11.5 (2022): 198-205.

- [7] Ramalingam, B., Lakshmanan, A. K., Ilyas, M., Le, A. V. & Elara, M. R. (2018). *Cascaded machine-learning technique for debris classification in floor-cleaning robot application*. Applied Sciences, 8(12), 2649.
- [8] Yin, J., Apuroop, K. G. S., Tamilselvam, Y. K., Mohan, R. E., Ramalingam, B., & Le, A. V. (2020). *Table cleaning task by human support robot using deep learning technique*. Sensors, 20(6), 1698.
- [9] R. Gopalakrishnan, U. Ramani, K. U. Maheswari and M. Thilagaraj, "Design and Development of Controller Based Automatic Ground Cleaning Robot," 2022 6th International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 2022.
- [10] Prassler, E., Ritter, A., Schaeffer, C. et al. *A Short History of Cleaning Robots*. Autonomous Robots 9, 211–226 (2000).
- [11] Sanchez-Iborra, Ramon, and Antonio F. Skarmeta. "Tinyml-enabled frugal smart objects: Challenges and opportunities." IEEE Circuits and Systems Magazine 20.3 (2020): 4-18.
- [12] Rakib, Abdul Fattah, et al. "A lightweight quantized cnn model for plant disease recognition." Arabian Journal for Science and Engineering 49.3 (2024): 4097-4108.
- [13] A. Paul, N. Rakhaine, N. J. Ohee and A. Ahammad, "A Comprehensive Android App Based Solution for Automated Attendance and Management in Institutions Using IoT and TinyML," 2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD), Dhaka, Bangladesh, 2023
- [14] K. Dong, C. Zhou, Y. Ruan and Y. Li, "MobileNetV2 Model for Image Classification," 2020 2nd International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, 2020, pp. 476-480, doi: 10.1109/ITCA52113.2020.00106.
- [15] Gulzar, Yonis. "Fruit image classification model based on MobileNetV2 with deep transfer learning technique." Sustainability 15.3 (2023): 1906.

- [16] Indraswari, Rarasmaya, Rika Rokhana, and Wiwiet Herulambang. "*Melanoma image classification based on MobileNetV2 network.*" *Procedia computer science* 197 (2022): 198-207
- [17] Warden, Pete, and Daniel Situnayake. *Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers.* O'Reilly Media, 2019.
- [18] Zim, Md Ziaul Haque. "*TinyML: analysis of Xtensa LX6 microprocessor for neural network applications by ESP32 SoC.*" arXiv preprint arXiv:2106.10652 (2021).
- [19] Mach, Václav, et al. "*Brushless DC motor control on arduino platform.*" *Przegląd elektrotechniczny* (2018).
- [20] Sudharsan, Bharath, Simone Salerno, and Rajiv Ranjan. "*TinyML-CAM: 80 FPS image recognition in 1 kB RAM.*" *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking.* 2022.